
****UNIT – IV**

Networking**

1. Networking Basics

Networking allows two or more devices to communicate and share data. Java provides a powerful `java.net` package to perform network operations such as communicating through sockets, accessing URLs, sending datagrams, etc.

Key Concepts

- **IP Address** – Unique address assigned to devices (e.g., 192.168.1.1).
- **Port Number** – Identifies a specific process on a machine.
- **Protocol** – Rules for communication (TCP, UDP, HTTP).
- **Sockets** – Endpoints for communication.
- **Client/Server model** – Server waits, Client connects.

2. Useful Classes & Interfaces (java.net package)

Important Classes

Class	Description
InetAddress	Represents IP Address (IPv4/IPv6).
Socket	TCP client socket.
ServerSocket	Listens for TCP client requests.
DatagramSocket	UDP communication endpoint.
DatagramPacket	Holds UDP data.
URL	Represents a resource on the internet.
URLConnection	Communication link between app and URL.
HttpURLConnection	Specialized URL connection for HTTP.

3. InetAddress

Common Methods

- `getLocalHost()`
- `getByName(String host)`
- `getHostAddress()`
- `getHostName()`

4. TCP (Transmission Control Protocol)

Features

- **Connection-oriented**
- **Reliable** (acknowledgement, retransmission)
- **Stream-based**
- **Slower but accurate**

TCP/IP PROGRAMS

TCP Server Program (Java)

```
import java.io.*;
import java.net.*;

public class TCPServer {
    public static void main(String[] args) {
        try {
            ServerSocket ss = new ServerSocket(5000);
            System.out.println("Server Started... Waiting for client!");

            Socket s = ss.accept();
            System.out.println("Client Connected.");

            BufferedReader br = new BufferedReader(new
InputStreamReader(s.getInputStream()));
            PrintWriter pw = new PrintWriter(s.getOutputStream(), true);

            String msg = br.readLine();
            System.out.println("Client Says: " + msg);

            pw.println("Hello Client, message received!");

            s.close();
            ss.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

```
}
```

TCP Client Program (Java)

```
import java.io.*;
import java.net.*;

public class TCPClient {
    public static void main(String[] args) {
        try {
            Socket s = new Socket("localhost", 5000);

            BufferedReader br = new BufferedReader(new
InputStreamReader(s.getInputStream()));
            PrintWriter pw = new PrintWriter(s.getOutputStream(), true);

            pw.println("Hello Server!");

            String reply = br.readLine();
            System.out.println("Server Says: " + reply);

            s.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

5. UDP (User Datagram Protocol)

Features

- **Connectionless**
 - **Fast**
 - **No guarantee of delivery (unreliable)**
 - **Message-based (Datagram)**
-

UDP PROGRAMS

UDP Server Program (Java)

```
import java.net.*;

public class UDPServer {
```

```

public static void main(String[] args) {
    try {
        DatagramSocket ds = new DatagramSocket(4000);
        byte[] buffer = new byte[1024];

        DatagramPacket dp = new DatagramPacket(buffer, buffer.length);
        System.out.println("Server waiting for data...");

        ds.receive(dp);

        String msg = new String(dp.getData(), 0, dp.getLength());
        System.out.println("Client Says: " + msg);

        String reply = "Message received by server";
        byte[] sendData = reply.getBytes();

        DatagramPacket dpSend = new DatagramPacket(
            sendData, sendData.length, dp.getAddress(), dp.getPort()
        );

        ds.send(dpSend);
        ds.close();
    } catch (Exception e) {
        System.out.println(e);
    }
}
}

```

UDP Client Program (Java)

```

import java.net.*;

public class UDPClient {
    public static void main(String[] args) {
        try {
            DatagramSocket ds = new DatagramSocket();
            InetAddress ip = InetAddress.getByName("localhost");

            String msg = "Hello UDP Server!";
            byte[] sendData = msg.getBytes();

            DatagramPacket dp = new DatagramPacket(sendData, sendData.length,
ip, 4000);
            ds.send(dp);

            byte[] buffer = new byte[1024];
            DatagramPacket dpReceive = new DatagramPacket(buffer,
buffer.length);
            ds.receive(dpReceive);

            String reply = new String(dpReceive.getData(), 0,
dpReceive.getLength());
            System.out.println("Server Says: " + reply);
        }
    }
}

```

```
        ds.close();
    } catch (Exception e) {
        System.out.println(e);
    }
}
}
```

6. URI, URL & URLConnection

URI (Uniform Resource Identifier)

- Identifies a resource (not necessarily accessible)
- Example: `mailto:abc@gmail.com`

URL (Uniform Resource Locator)

- Specifies location of resource
- Example: `https://example.com/index.html`

URLConnection

- Allows communication between Java program and URL

HttpURLConnection

- Used specifically for HTTP operations (GET, POST)
-

7. Datagrams

- Used in **UDP**
 - Units of independent data packets
 - Not guaranteed to arrive in order
 - Classes:
 - **DatagramSocket**
 - **DatagramPacket**
-

MAIN MUST-KNOW POINTS SUMMARY

General Networking

- IP address, port, host, protocol
- Client-server architecture
- Socket communication

TCP Important Points

- Reliable, connection-oriented
- Uses stream sockets (`Socket`, `ServerSocket`)
- Good for banking, file transfer, chat apps

UDP Important Points

- Fast, connectionless
- Uses datagram sockets
- Suitable for video streaming, games, VoIP

Java Networking Classes

- `InetAddress`
- `Socket`, `ServerSocket`
- `DatagramSocket`, `DatagramPacket`
- `URL`, `URLConnection`, `HttpURLConnection`

URL & HTTP

- URL components (protocol, host, port, path)
- GET vs POST

If you want, I can also prepare:

- ✓ A **PDF** with this content
- ✓ A **PowerPoint** formatted for exam presentation
- ✓ Flowcharts / diagrams for TCP and UDP communication

Just tell me!

You said:

what are the steps involved into it

ChatGPT said:

Here are the **steps involved in Networking (Unit-IV)** — clear, exam-ready, and structured.

STEPS INVOLVED IN JAVA NETWORKING

Below are the steps for TCP, UDP, and URL/URLConnection.

1. Steps in TCP Communication (Client-Server)

A. Server Side Steps (TCP Server)

1. **Create a ServerSocket**
`ServerSocket ss = new ServerSocket(port);`
 2. **Wait for client request (blocking)**
`Socket s = ss.accept();`
 3. **Create input and output streams**
 - o `InputStream / BufferedReader` → receive
 - o `PrintWriter / OutputStream` → send
 4. **Perform communication**
Exchange messages with the client.
 5. **Close connection**
Close socket and server socket.
-

B. Client Side Steps (TCP Client)

1. **Create a socket to connect to server**
`Socket s = new Socket(host, port);`
 2. **Create input and output streams**
Send to server → Output stream
Receive from server → Input stream
 3. **Exchange messages**
 4. **Close the socket**
-

2. Steps in UDP Communication (Client–Server)

A. Server Side Steps (UDP Server)

1. **Create DatagramSocket**
`DatagramSocket ds = new DatagramSocket(port);`
 2. **Create buffer and packet to receive data**
`DatagramPacket dp = new DatagramPacket(buffer, buffer.length);`
 3. **Receive packet**
`ds.receive(dp);`
 4. **Process data**
Convert `byte[]` → `String`.
 5. **Send response back**
Create new `DatagramPacket` with client address/port.
 6. **Close socket**
-

B. Client Side Steps (UDP Client)

1. **Create DatagramSocket**
`DatagramSocket ds = new DatagramSocket();`
 2. **Convert data to byte[]**
 3. **Create DatagramPacket (send)**
Include IP + port.
 4. **Send packet**
`ds.send(packet);`
 5. **Create receive packet and wait for reply**
 6. **Close socket**
-

3. Steps in Using URL & URLConnection

A. Working with URL

1. **Create URL object**
`URL url = new URL("https://example.com");`
2. **Open a stream**
`InputStream is = url.openStream();`
3. **Read data**
4. **Close stream**

B. Using URLConnection

1. **Create URL object**
 2. **Open connection**
`URLConnection conn = url.openConnection();`
 3. **Connect implicitly**
When using streams.
 4. **Get InputStream / OutputStream**
 5. **Read or send data**
 6. **Close streams**
-

4. Steps in Using HttpURLConnection

1. **Create URL object**
 2. **Open HttpURLConnection**
`HttpURLConnection con = (HttpURLConnection) url.openConnection();`
 3. **Set request method (GET/POST)**
 4. **Send request (for POST)**
Write data using OutputStream.
 5. **Get response code**
 6. **Read response**
 7. **Close streams & disconnect**
-

★ Summary: MUST KNOW STEPS

Area	Steps
TCP Server	Create ServerSocket → accept() → streams → communicate → close
TCP Client	Create Socket → streams → communicate → close
UDP Server	DatagramSocket → receive → process → send → close
UDP Client	DatagramSocket → send → receive → close
URL	URL → openStream → read → close
URLConnection	URL → openConnection → get streams → communicate
HttpURLConnection	URL → open → set method → send/receive → close